

# Edge AI — putting the model on the ESP32

200 KB of RAM, no NumPy, and a new window every 128 milliseconds

- Edge, cloud, hybrid — and which row of the table actually decides it
- The compute budget: measuring every stage against the window duration
- A 256-point FFT in plain Python, and whether it earns its place
- Exporting a model: folding the scaler away, and what does not export at all
- Single precision, heap allocation, and code that survives a year of running
- Fail-safe behaviour, and a fault neither model was ever trained on

---

Notebook: `iaiiot26_Lect11_nb.ipynb`

# Edge, cloud, or both

- Three architectures, four rows of comparison — and only one row is a genuine engineering constraint rather than a preference.

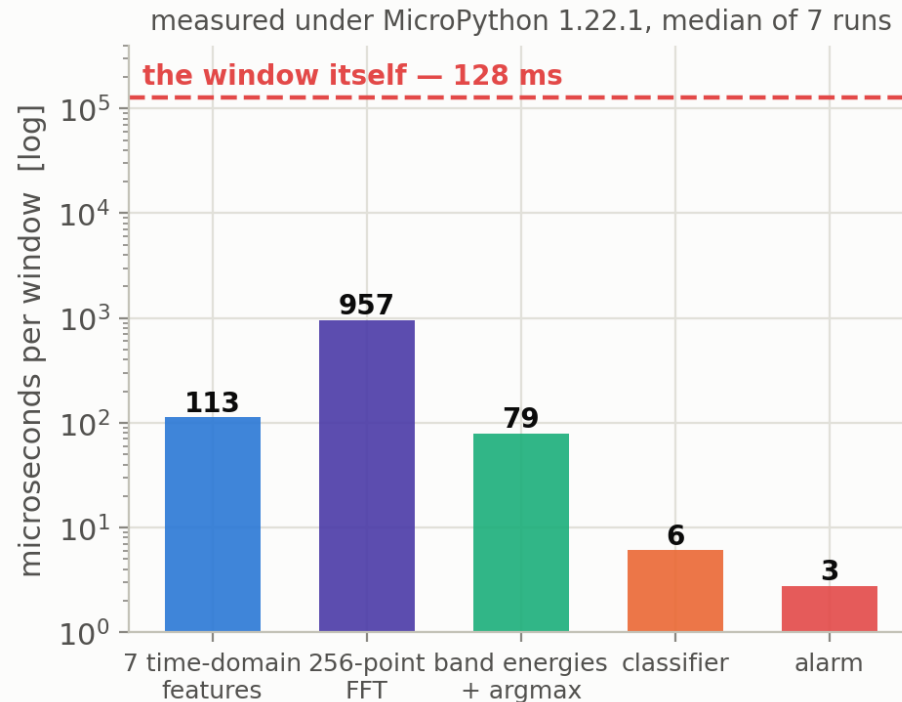
|                      | everything<br>on the device | everything<br>in the cloud         | hybrid<br>— this course                 |
|----------------------|-----------------------------|------------------------------------|-----------------------------------------|
| latency              | one window,<br>128 ms       | plus a round trip<br>to the broker | local decision,<br>cloud second opinion |
| bandwidth            | 12 numbers<br>per window    | every sample,<br>2000 per second   | features up,<br>not waveforms           |
| if the network drops | keeps deciding              | blind                              | keeps deciding,<br>reports later        |
| model you can run    | small, fixed                | anything you like                  | small local,<br>large remote            |

*The hybrid column is not a compromise — it is the only column where the machine is still protected when the WiFi is not.*

Bandwidth is a cost you can pay. Latency is a delay you can usually tolerate. Deciding correctly while the network is down is neither — it is the requirement, and it is the reason the decision runs on the device.

# The budget is set by the sensor

- Every stage timed under real MicroPython 1.22.1, median of 7 runs — and the ratios, not the microseconds, are what transfer to your board.

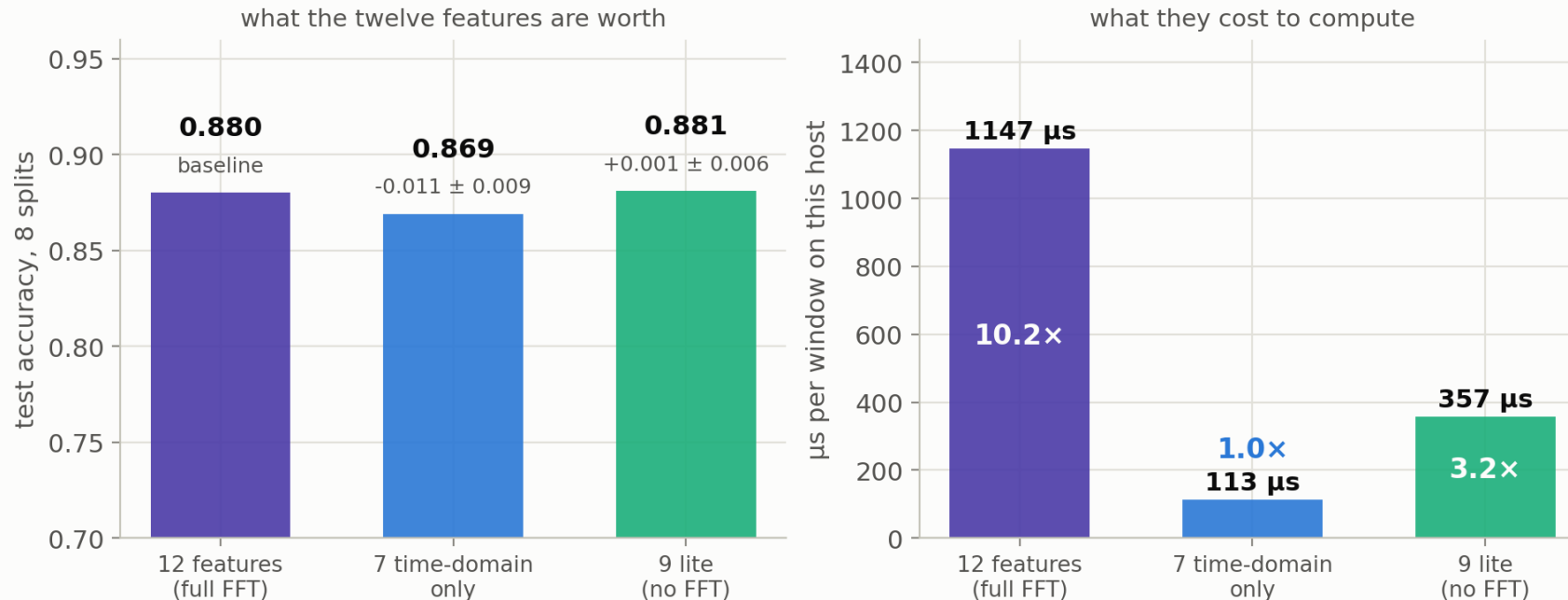


*This host is a desktop MicroPython build and is far faster than an ESP32. The RATIOS transfer; the absolute numbers do not.*

**DESIGN RULE** • if your Lab 8 measurement of the seven time-domain features is under 12.5 ms, the full twelve-feature pipeline fits the 128 ms window. Between 12.5 and 40.0 ms, take the lite path.

# Does the FFT earn its place?

- The FFT costs 8.5× the time-domain features. Retraining on three feature sets over the same eight splits says what it buys.

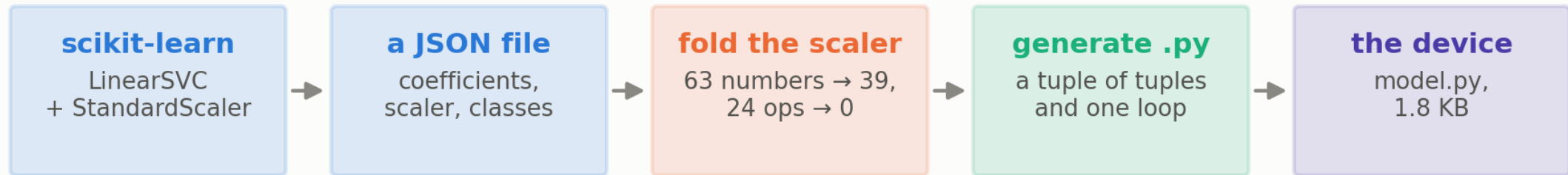


*The twelve features cost 10× the seven time-domain ones and buy  $+0.011 \pm 0.009$  of accuracy. Two cheap substitutes match it within noise, for a third of the cost.*

**A cheap, uninformative feature is an easy decision. The hard case is expensive and mildly informative — and you cannot tell which you are in without measuring both halves.**

# Exporting a model

- A fitted model is a data structure, not a program. Exporting means writing down its arithmetic — and not every model has any.



## exports to a few hundred bytes

linear models · decision trees · anything whose prediction is a fixed arithmetic expression

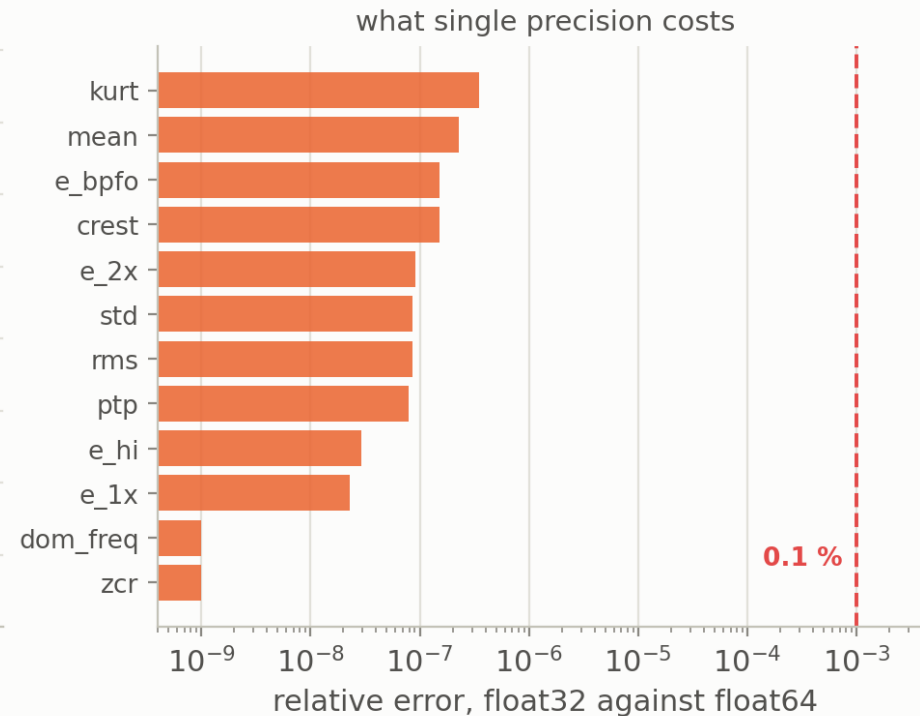
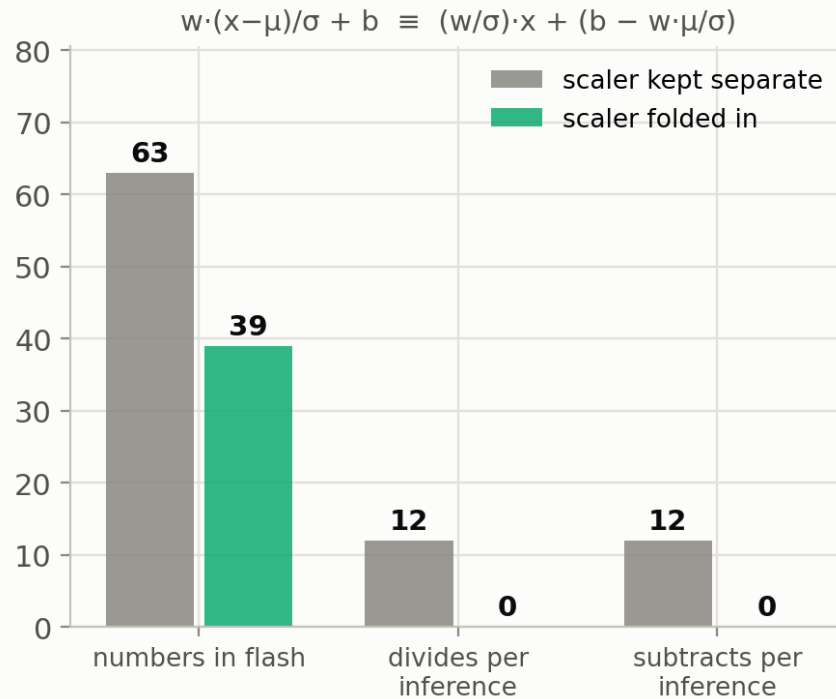
## does not export

k-NN — it IS the training set · random forests — hundreds of trees · anything that allocates while predicting

Generate the device file from the artefact with a script. Never transcribe a coefficient by hand — retraining must regenerate model.py, or the two will silently disagree.

# Two tricks that cost nothing

- Folding the standardiser into the coefficients removes 24 numbers and 24 operations per inference — exactly, because it is algebra.

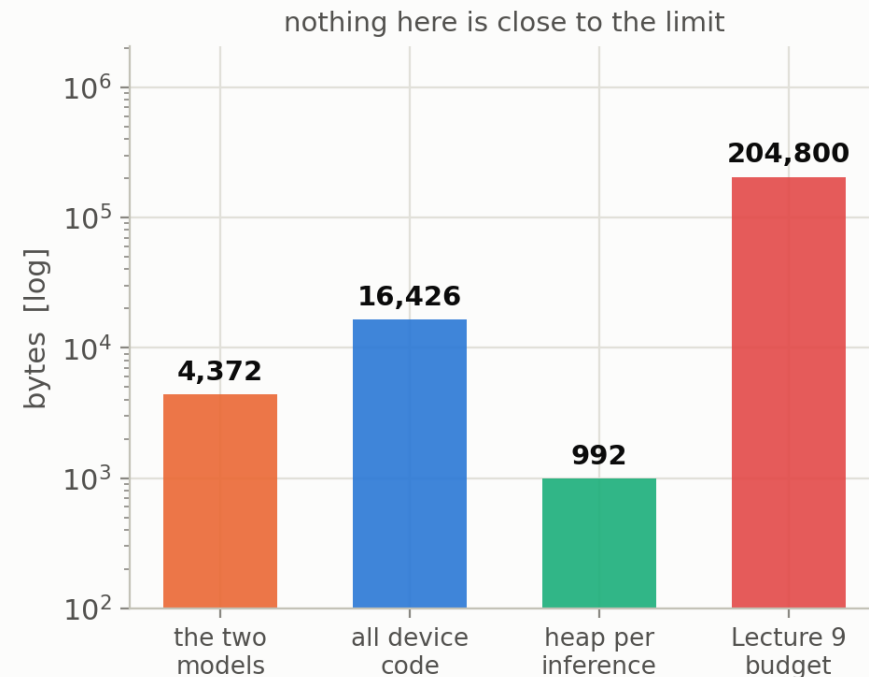
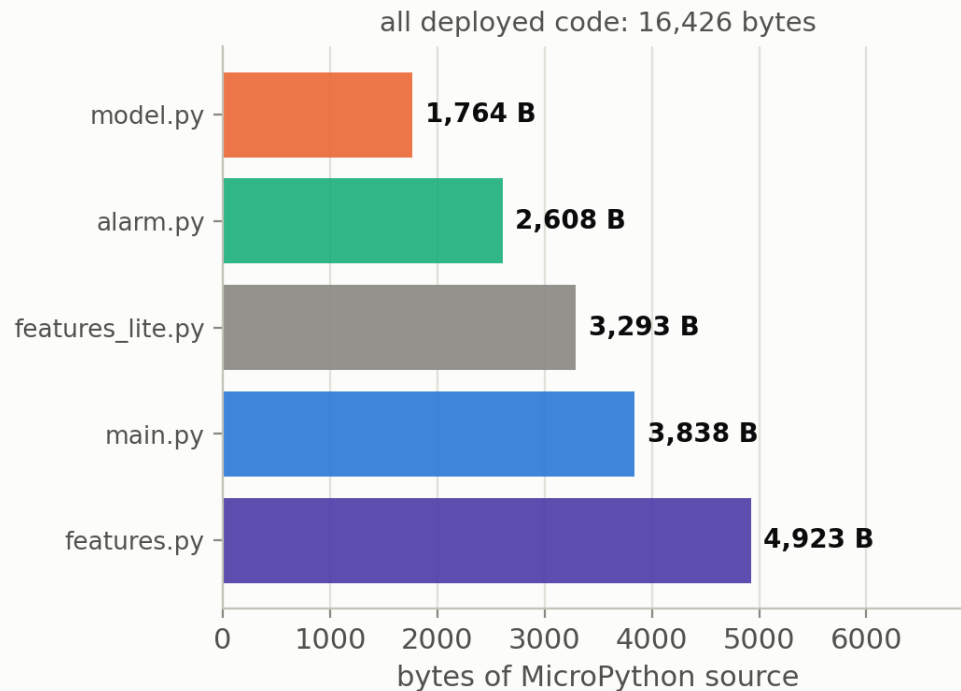


*Worst feature is kurt at 3e-07 — which is why Lab 8 asked for agreement to three decimals, not four.*

$w \cdot (x - \mu) / \sigma + b \equiv (w/\sigma) \cdot x + (b - w \cdot \mu / \sigma)$ . The standardiser does not get optimised on the device — it ceases to exist.

# Does it fit?

- The entire edge pipeline — features, FFT, fallback, both models, main loop — is 16.0 KB of source and allocates 992 bytes per inference.

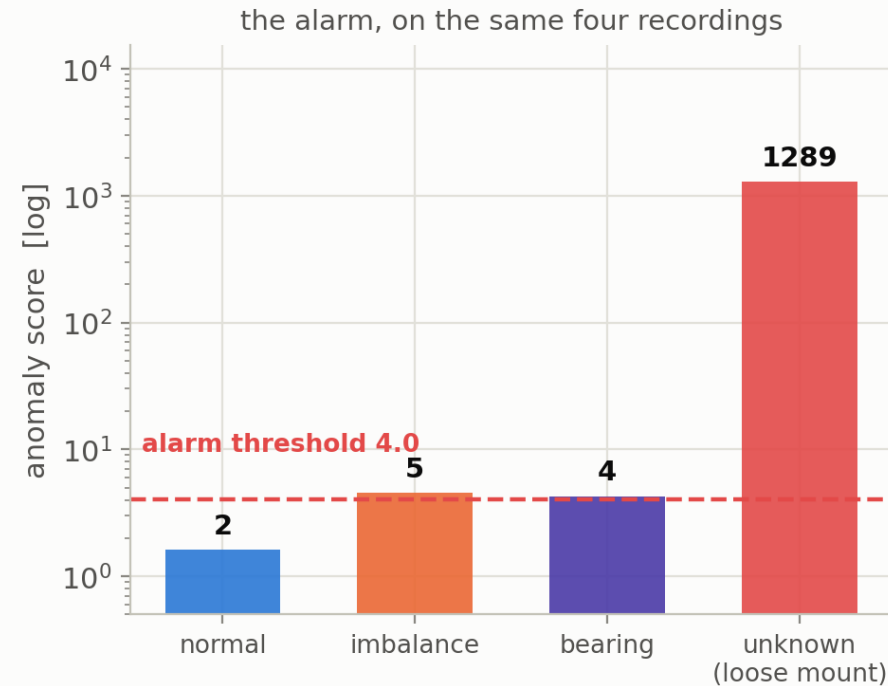
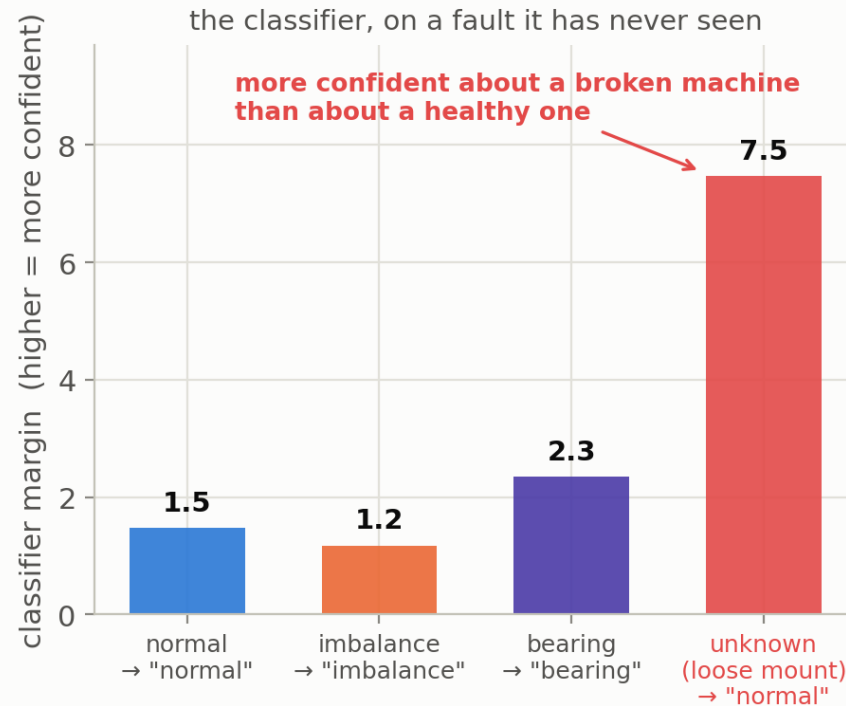


*One inference allocates 992 bytes of heap. The FFT buffers are allocated once at import, not per window — that is the difference between a device that runs for a year and one that does not.*

**The heap number matters more than the flash number, and it matters because it is constant. Free-running embedded code is judged on allocation per iteration, not on peak memory.**

# A fault the classifier has never seen

- A loose mounting bolt — a state in no training set anywhere in this course. The classifier has three labels and no way to say 'none of these'.

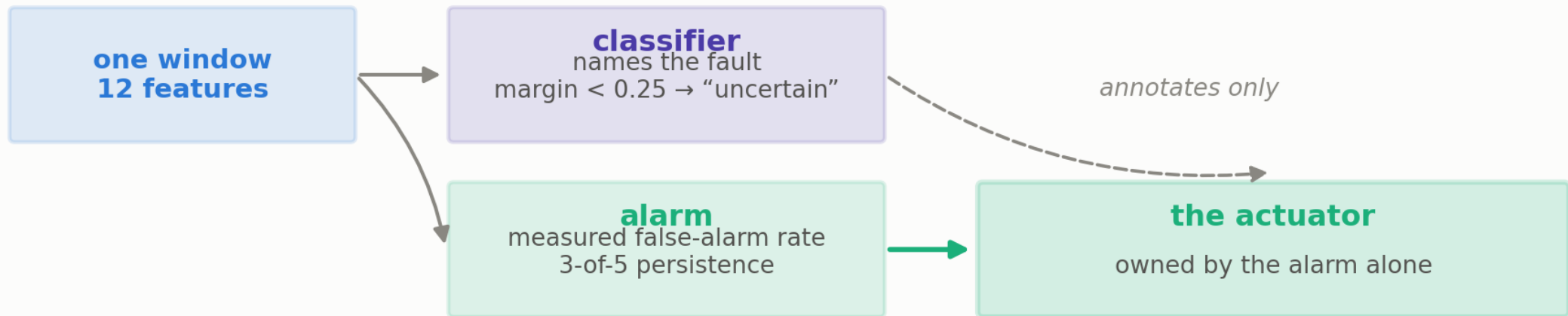


The classifier was never trained on a loose mounting bolt, so it has no way to be unsure about one. The alarm was never trained on anything, so it notices.

The classifier calls it normal with margin 7.5 — 5× its margin on a healthy machine. The alarm, which was never trained on any fault, raises on 93 % of windows.

# Who owns the actuator

- The alarm acts; the classifier annotates — because the alarm's false-alarm rate was measured, and the classifier's confidence was not.



## and when something breaks

- features raise → hold the last state, log it, do not guess
- model raises → the alarm still runs; the label becomes "model-error"
- network is down → nothing changes: no decision here needs it

The last row is the argument for edge AI in one line: every decision here is made from twelve numbers the device computed itself. The network carries the report, not the decision.

# The whole device, in one loop

- Read, window, twelve features, two models, one actuator decision — with a timing check that tells you immediately if the board cannot keep up.

```
while True:
    buf = [read_sample() for _ in range(256)]    # 128 ms of signal
    t0 = time.ticks_us()

    try:
        x = features.all_features(buf)           # 12 numbers
    except Exception:
        continue                                # hold, log, do not guess

    label, margin = classify(x)                   # names the fault
    raised, score = alarm.update(x)               # decides what to do
    _, driver      = alarm.worst_feature(x)        # which feature fired

    if raised != raised_last:                    # only the alarm acts
        set_actuator(raised)
        raised_last = raised

    if time.ticks_diff(time.ticks_us(), t0) > WINDOW_US:
        print('OVER BUDGET')                     # you cannot keep up
```

**Measured on this host: 1156  $\mu$ s per window against a 128 ms budget — 0.9 % used. Lab 11 asks you for the same number from the ESP32, and that is the only one you may quote.**

# Lab 11 — the edge pipeline on the ESP32

## A • measure

Run `bench.py` on the ESP32 in Wokwi. Fill in the stage table and compute your own two ratios.

Does the full twelve-feature pipeline fit the 128 ms window on your board? By what margin?

## B • deploy

Flash `main.py` with the exported model and alarm. Run the three known waveforms and check the labels.

Verify your features against `expected_features.csv` to at least three decimals before trusting any of it.

## C • break it

Switch WAVE to `wave_unknown` — the loose mounting bolt. Record what each model says.

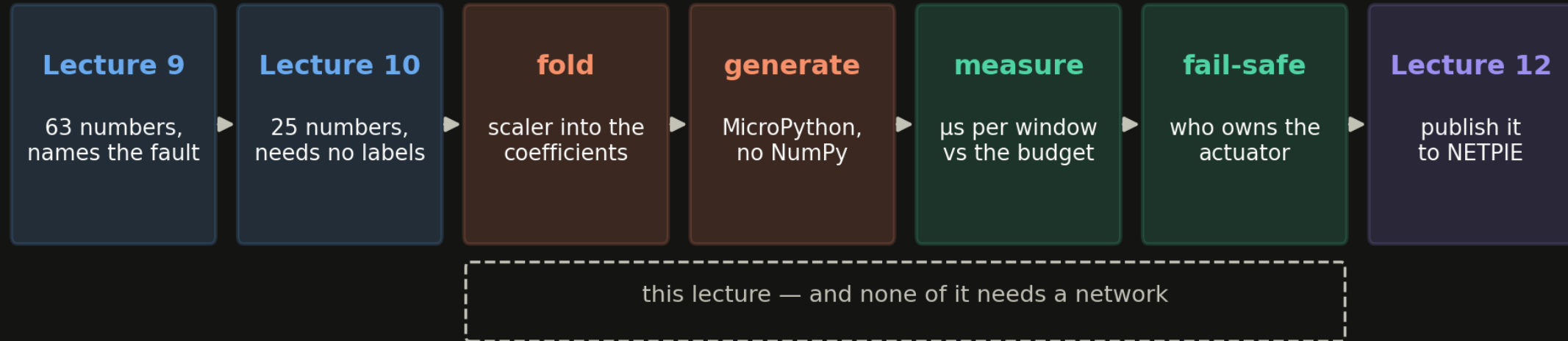
Then choose: keep the FFT, or justify the lite path with your own measurement.

```
$ cp wokwi/*.py <your Wokwi project>
$ # bench.py first, main.py second

win,label,margin,anom_score,driver,raised,us
0,normal,7.5,1289.4,dm_freq,0,1214
...
# mean 1156 us per window, budget 128000 us
```

**Deliverable:** your measured stage table, the pass/fail against the window budget, and one paragraph on what the device should do about a fault it cannot name.

# Features go up, decisions stay local



1. Verify the device features against NumPy before you time anything. Fast and wrong is worse than slow and right.
2. Measure on the target. The ratios transfer (FFT 8.5×, pipeline 10.2×); the microseconds do not.
3. Fold the scaler into the coefficients: 63 numbers become 39, and 24 operations become none.
4. Check single precision against the margin you depend on — worst here was kurt at 3e-7.
5. Allocate every buffer once at import. 992 bytes per pass, and constant, is what runs for a year.
6. Run both models. The classifier names what it knows; the alarm noticed a fault it called normal with margin 7.5.

Generated by Claude and customized by

